

单片机程序

mian.c

启动单片机外设

- GPIO — 给SPI做片选引脚
- DMA — 配置了但是好像没用上
- SPI — 和BMI088通信
- FDCCAN — 和电机的通信协议
- PWM — 加在加热电阻上稳定BMI088的温漂 — 配置了没用上
- DWT — 这个猛，直接对CPU周期进行计数，不是普通定时器 — 提供高精度时间基准，控制时间间距dt的计算

初始化IMU和CAN

BMI088 — 运行流程（上到下）：

- 原始输出：
 - 三轴加速度
 - 三轴角速度
 - 温度
- 程序处理：
 - 首先BMI088_Read()读取原始数据
 - 原始数据送进Mahony算法算姿态角

启动RTOS

RTOS

```
/* Call init function for freertos objects (in freertos.c) */
void FREERTOS_init()
/* Start scheduler */
osKernelStart();
```

任务定义与创建
定义参数：任务名字，函数入口，优先级，实例数（一般给0不用管），栈大小
osKernelStart()之后这些任务就开始调度

一共定义了6个任务

defaultTask
空的，啥也不是，不知道为啥32这块预留这个东西

INS_TASK（Inertial Navigation System）
注意到这个优先级最高
任务内容：
1.读取IMU原始数据
2.解算得到PRY
3.得到去掉重力分量的两个坐标系上的加速度

CHASSISR_TASK（Chassis：底盘）
剩下三个优先级一样，注意放线
任务内容：
1.初始化右侧电机ID和MIT模式
2.读取右侧关节角反馈
3.计算右腿五连杆状态 L0, phi0, theta, dtheta
4.根据腿长L0 调度 LQR_K
5.计算右轮 wheel_T
6.计算右腿虚拟力矩 Tp
7.计算右腿轴向力 F0
8.加入 roll/yaw/防劈叉补偿
9.VMC 映射成右髋两个电机电矩
10.通过 CAN1 发给右侧三个电机

CHASSISL_TASK

OBSERVE_TASK
这个第二高
任务内容：
1.估计 xdot 和 x

PS2_TASK
功能：
1.开机/关机
2.倒地自起
3.右摇杆X控制yaw转向
4.右摇杆Y控制前进速度
5.左摇杆X控制roll：kunkun跳舞
6.左摇杆Y控制腿长
7.L2键让rollset归零（合理）

这6个任务之间互相通信的方式是共同访问同一批全局结构体，比较原始

首先区别地面系和机体系
地面系：坐标轴固定在地面上，坐标系不跟着机器人歪
机体系：坐标轴焊在机器人上，机器人怎么歪坐标系就怎么歪
可以看出，传感器给的数据是机体系上面的，而xdoot是地面系的
用轮子速度+腿部角速度项+腿长速度项得到对xdot的主要估计（就是基于模型运动学的估计）
然后用IMU去掉重力的加速度辅助估计，讲道理更适合用地面系，但是达妙用的机体系。
两种数据来源送进卡尔曼
当轮子打滑的时候不相信动力学算出来的轮速
当机体俯仰角度剧烈，或者传感器距离轮子很远的时候不相信IMU的加速度